Name: **Shrey Pendurkar**
Class: **D15C**
Batch: **B**
Roll No: **38**

MLDL – Experiment 4

Aim

Implement K-Nearest Neighbors (KNN) and evaluate model performance.

---

1. Dataset Source

- Dataset: Iris Classification
- Kaggle: https://www.kaggle.com/datasets/sachgarg/iris-classification
- File used: Iris.csv

---

2. Dataset Description

The Kaggle Iris Classification dataset is based on the classic Fisher Iris dataset and contains four physical measurements of iris flowers along with their species labels. It is a balanced multi-class classification benchmark with three target classes.

Columns in Iris.csv:

- Id: Row identifier — dropped before modelling (non-informative)
- SepalLengthCm: Sepal length in centimetres (numeric)
- SepalWidthCm: Sepal width in centimetres (numeric)
- PetalLengthCm: Petal length in centimetres (numeric)
- PetalWidthCm: Petal width in centimetres (numeric)
- Species: Target label — three classes: Iris-setosa, Iris-versicolor, Iris-virginica

Key properties:

- Number of samples: 150
- Number of features: 4 numeric features (after dropping Id)
- Target variable: Species (3 classes, 50 samples each — perfectly balanced)
- Missing values: None
- Goal: Predict the Iris species from the four measurement features

---

3. Mathematical Formulation of the Algorithm

K-Nearest Neighbors (KNN) is a non-parametric, instance-based supervised learning algorithm. It does not learn explicit model parameters; instead, it classifies new samples by examining the K most similar training samples.

3.1 Distance Metric

For a query sample x and a training sample x_i in an m-dimensional feature space, the Euclidean distance is:

$$d(x, x_i) = \sqrt{\left(\Sigma_{j=1}^{m} (x_j - x_{ij})^2 \right)}$$

KNN uses Euclidean distance (Minkowski distance with p=2) by default for continuous features.

3.2 Prediction Rule (Majority Vote)

Given a new sample x:

- Compute the distance between x and every training sample
- Select the K training samples with the smallest distances (the K nearest neighbors)
- Let their class labels be y_1 , y_2 , …, y^T
- Predict the class by majority voting:

$$\hat{y} = argmax_c \ \Sigma_{i=1}^{T} \ \square(y_i = c)$$

where $\square(\cdot)$ is the indicator function that equals 1 if the condition is true and 0 otherwise.

3.3 Distance-Weighted Voting (optional variant)

Closer neighbors can be given more influence by weighting each vote by the inverse of its distance. The uniform-weight variant (used as default) treats all K neighbors equally.

---

4. Algorithm Limitations

1. **High Prediction Time**

   KNN stores all training samples and computes distances to every one of them at prediction time. This makes inference slow on large datasets — O(n) per query — unlike parametric models that amortise computation into a fixed-size parameter vector during training.

2. **Curse of Dimensionality**

   As the number of features grows, Euclidean distances become increasingly uniform across all training samples, making it harder to identify meaningful nearest neighbors. KNN requires careful feature selection or dimensionality reduction on high-dimensional data.

3. **Sensitivity to Feature Scaling**

Features with larger numeric ranges dominate the distance calculation. For instance, an unscaled area feature (in thousands) would overshadow a binary feature. StandardScaler normalization is therefore critical before applying KNN.

4. Choice of K

Very small K (e.g., K=1) leads to overfitting — the model memorises training points and is highly sensitive to noise (high variance). Very large K leads to underfitting — the decision boundary becomes overly smooth (high bias). K must be tuned empirically.

5. Sensitivity to Noise and Class Imbalance

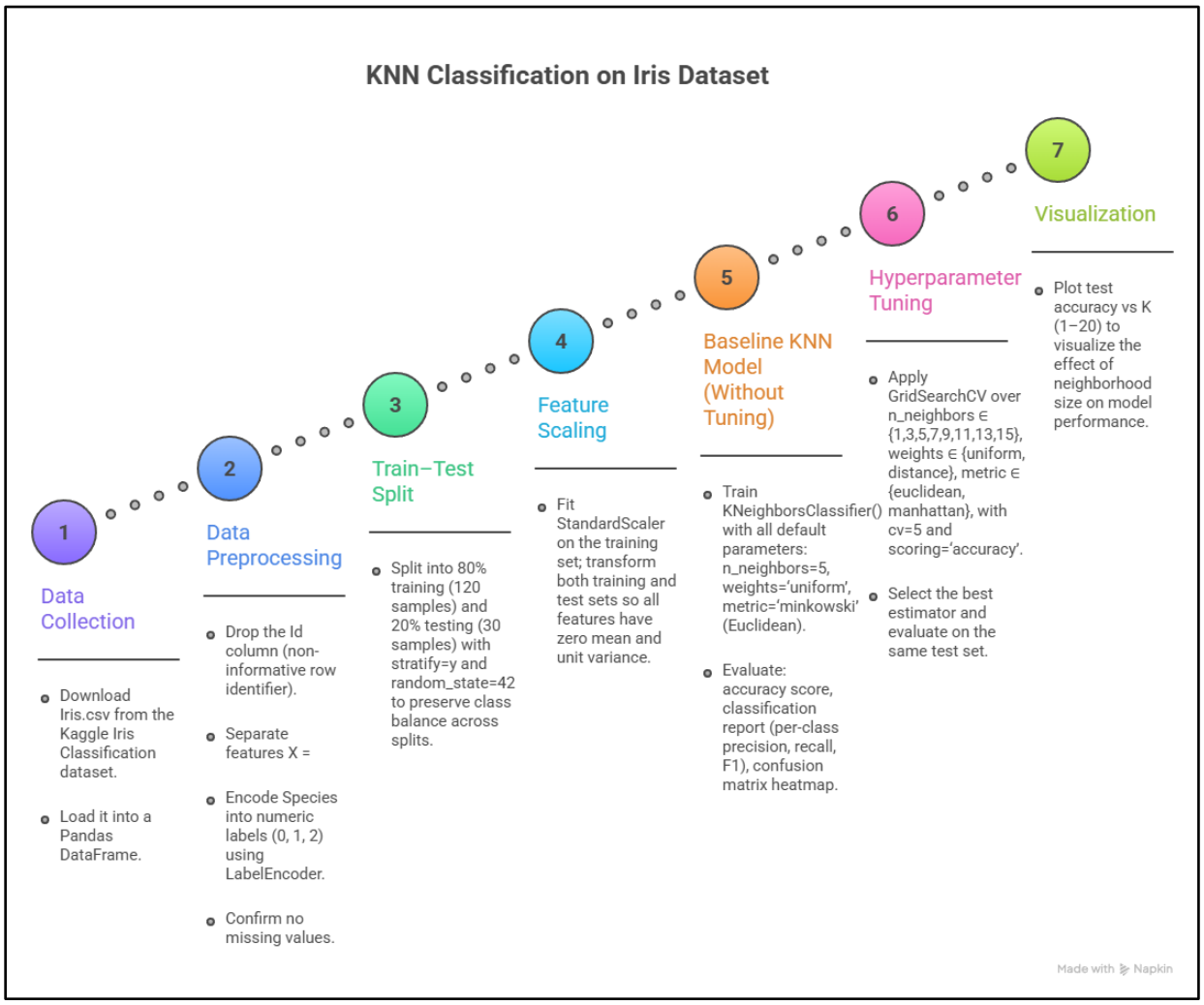
Noisy training points or imbalanced class distributions can mislead majority voting near decision boundaries, particularly when the minority class is sparse in local neighborhoods.

---

5. Methodology / Workflow

1. **Data Collection**
   - Download Iris.csv from the Kaggle Iris Classification dataset.
   - Load it into a Pandas DataFrame.
2. **Data Preprocessing**
   - Drop the Id column (non-informative row identifier).
   - Separate features X = [SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm] and target y = Species.
   - Encode Species into numeric labels (0, 1, 2) using LabelEncoder.
   - Confirm no missing values.
3. **Train–Test Split**
   - Split into 80% training (120 samples) and 20% testing (30 samples) with stratify=y and random_state=42 to preserve class balance across splits.
4. **Feature Scaling**
   - Fit StandardScaler on the training set; transform both training and test sets so all features have zero mean and unit variance.
5. **Baseline KNN Model (Without Tuning)**
   - Train KNeighborsClassifier() with all default parameters: n_neighbors=5, weights='uniform', metric='minkowski' (Euclidean).
   - Evaluate: accuracy score, classification report (per-class precision, recall, F1), confusion matrix heatmap.
6. **Hyperparameter Tuning**
   - Apply GridSearchCV over n_neighbors ∈ {1,3,5,7,9,11,13,15}, weights ∈ {uniform, distance}, metric ∈ {euclidean, manhattan}, with cv=5 and scoring='accuracy'.
   - Select the best estimator and evaluate on the same test set.

7. **Visualization**
   - Plot test accuracy vs K (1–20) to visualize the effect of neighborhood size on model performance.



---

6. Performance Analysis

Model: KNeighborsClassifier() — all default parameters (n_neighbors=5, weights='uniform', metric='euclidean')

Train/Test Split: 80% / 20% | Test Set Size: 30 samples (10 per class, stratified)

Metric	Iris-setosa	Iris-versicolor	Iris-virginica	Overall
Precision	1.00	0.83	1.00	—
Recall	1.00	1.00	0.80	—

F1-Score	1.00	0.91	0.89	—
Support	10	10	10	30
Accuracy	—	—	—	93.33%

1. **Iris-setosa**

   The model perfectly identifies all setosa samples (precision 1.00, recall 1.00, F1 1.00). Setosa is linearly separable from the other two species in petal length and petal width, making it trivially easy for a distance-based classifier to distinguish.

2. **Iris-versicolor**

   Versicolor shows perfect recall (1.00) — all 10 true versicolor samples are correctly identified. However, precision drops to 0.83, indicating that some virginica samples are misclassified as versicolor (false positives). This reflects the known overlap between these two species in feature space.

3. **Iris-virginica**

   Virginica achieves perfect precision (1.00) but recall of 0.80 — 2 out of 10 true virginica samples are misclassified as versicolor (false negatives). The model is conservative but highly accurate when it does predict virginica.

4. **Overall**

   The baseline KNN (K=5, Euclidean, uniform) correctly classifies 28 out of 30 test samples (93.33%). Both misclassifications occur at the versicolor–virginica boundary, consistent with the natural overlap between these species.

---

7. Hyperparameter Tuning

Before Tuning — Default Parameters

Parameter	Default Value
n_neighbors	5
weights	uniform
metric	euclidean (minkowski, p=2)

Baseline Test Accuracy: 93.33%

GridSearchCV Configuration:

- n_neighbors: [1, 3, 5, 7, 9, 11, 13, 15]
- weights: ['uniform', 'distance']
- metric: ['euclidean', 'manhattan']
- cv = 5, scoring = 'accuracy', n_jobs = −1
- Total combinations evaluated: 8 × 2 × 2 = 32

Best Parameters Found:

Parameter	Best Value
n_neighbors	5
weights	uniform
metric	euclidean

Best Cross-Validation Accuracy: 96.67%

After Tuning — Results:

Metric	Iris-setosa	Iris-versicolor	Iris-virginica	Overall
Precision	1.00	0.83	1.00	—
Recall	1.00	1.00	0.80	—
F1-Score	1.00	0.91	0.89	—
Support	10	10	10	30
Accuracy	—	—	—	93.33%

Key Observations:

- Test accuracy remains 93.33% before and after tuning — GridSearchCV confirmed that the default configuration is already optimal for this dataset
- The best parameters (K=5, euclidean, uniform) are identical to the sklearn defaults, providing strong statistical evidence that these settings generalise well
- The CV accuracy (96.67%) is higher than the test accuracy (93.33%) — with only 30 test samples (10 per class), a single misclassification changes accuracy by ~3.3%, so a small test set can underestimate true generalisation
- Distance weighting and Manhattan metric did not improve performance on this dataset — the four features are already well-scaled and Euclidean distance captures the structure of the data effectively
- The remaining 2 misclassifications (both at the versicolor–virginica boundary) reflect irreducible class overlap in the dataset, not a shortcoming of the algorithm or its configuration

---

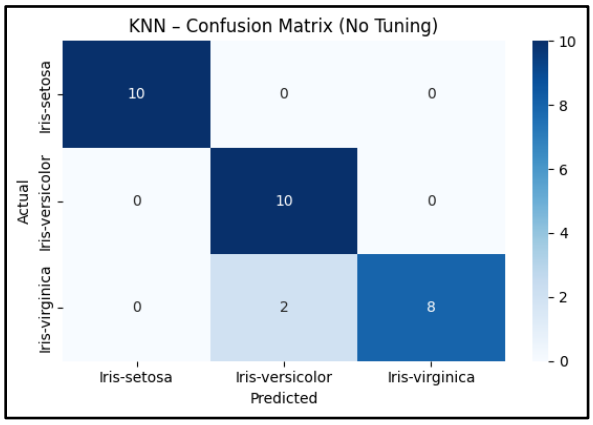
8. Code and Output – Without Tuning

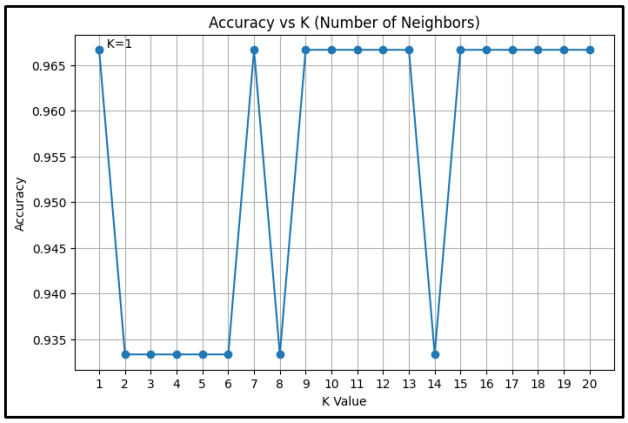
```
# K-Nearest Neighbors – Without Hyperparameter Tuning
# Dataset: Kaggle Iris Classification (Iris.csv)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import
train_test_split
from sklearn.preprocessing import
StandardScaler, LabelEncoder
from sklearn.neighbors import
KNeighborsClassifier
from sklearn.metrics import
accuracy_score, classification_report,
confusion_matrix

# Load dataset
df = pd.read_csv('Iris.csv')
print("First 5 rows:")
print(df.head())
print("\nShape:", df.shape)

# Drop non-informative Id column
if 'Id' in df.columns:
    df.drop(columns=['Id'], inplace=True)

# Features and target
feature_cols = ['SepalLengthCm',
'SepalWidthCm', 'PetalLengthCm',
'PetalWidthCm']
X = df[feature_cols]
y = df['Species']

# Encode target labels
le = LabelEncoder()
y_encoded = le.fit_transform(y)
print("\nClass mapping:",
dict(zip(le.classes_,
le.transform(le.classes_))))

# Train-test split (80/20, stratified)
X_train, X_test, y_train, y_test =
train_test_split(
    X, y_encoded, test_size=0.2,
random_state=42, stratify=y_encoded
)
print("\nTrain shape:", X_train.shape, "|
Test shape:", X_test.shape)

# Feature scaling
scaler = StandardScaler()
X_train_scaled =
scaler.fit_transform(X_train)
X_test_scaled  =
scaler.transform(X_test)

# Train baseline KNN (all defaults:
n_neighbors=5, uniform weights,
Euclidean)
knn = KNeighborsClassifier()
knn.fit(X_train_scaled, y_train)

# Predictions
y_pred = knn.predict(X_test_scaled)

# Performance
acc = accuracy_score(y_test, y_pred)
print("\n========== KNN (No Tuning)
==========")
print(f"Accuracy: {acc * 100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test,
y_pred, target_names=le.classes_))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d',
cmap='Blues',
            xticklabels=le.classes_,
yticklabels=le.classes_)
plt.title('KNN – Confusion Matrix (No
Tuning)')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
plt.show()

print("\nDefault Parameters Used:")
print(knn.get_params())

# ================= Accuracy vs K
Plot =================
k_values = range(1, 21)  # K from 1 to
20
accuracies = []
```

```
for k in k_values:
    knn_k =
KNeighborsClassifier(n_neighbors=k)
    knn_k.fit(X_train_scaled, y_train)
    y_pred_k =
knn_k.predict(X_test_scaled)
    acc_k = accuracy_score(y_test,
y_pred_k)
    accuracies.append(acc_k)

# Find best K
best_k =
k_values[np.argmax(accuracies)]
best_acc = max(accuracies)

print(f"\nBest K: {best_k} with Accuracy:
{best_acc * 100:.2f}%")
```

```
# Plot Accuracy vs K
plt.figure(figsize=(8, 5))
plt.plot(k_values, accuracies,
marker='o')
plt.title('Accuracy vs K (Number of
Neighbors)')
plt.xlabel('K Value')
plt.ylabel('Accuracy')
plt.xticks(k_values)
plt.grid(True)

# Highlight best K
plt.scatter(best_k, best_acc)
plt.text(best_k, best_acc, f'  K={best_k}',
verticalalignment='bottom')

plt.show()
```



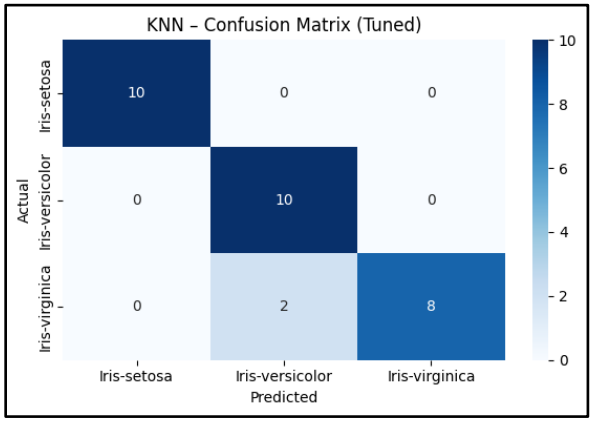


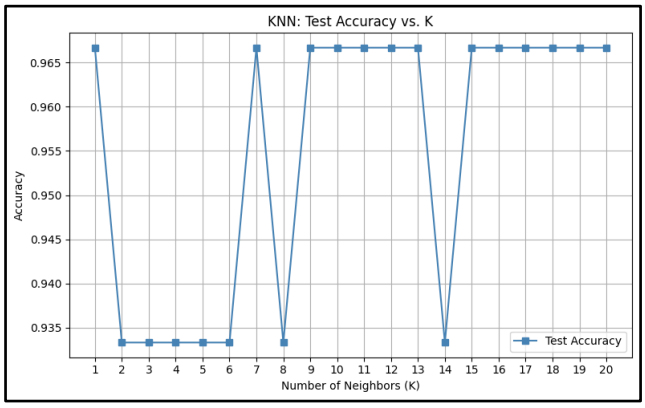
9. Code and Output – With Tuning

```
# K-Nearest Neighbors – With
Hyperparameter Tuning (GridSearchCV)
# Dataset: Kaggle Iris Classification
(Iris.csv)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import
train_test_split, GridSearchCV
from sklearn.preprocessing import
StandardScaler, LabelEncoder
```

```
from sklearn.neighbors import
KNeighborsClassifier
from sklearn.metrics import
accuracy_score, classification_report,
confusion_matrix

# Load dataset
df = pd.read_csv('Iris.csv')
if 'Id' in df.columns:
    df.drop(columns=['Id'], inplace=True)
```

```
feature_cols = ['SepalLengthCm',
'SepalWidthCm', 'PetalLengthCm',
'PetalWidthCm']
X = df[feature_cols]
y = df['Species']

le = LabelEncoder()
y_encoded = le.fit_transform(y)

# Same split as baseline
X_train, X_test, y_train, y_test =
train_test_split(
    X, y_encoded, test_size=0.2,
random_state=42, stratify=y_encoded
)

scaler = StandardScaler()
X_train_scaled =
scaler.fit_transform(X_train)
X_test_scaled  =
scaler.transform(X_test)

# Hyperparameter grid
param_grid = {
    'n_neighbors': [1, 3, 5, 7, 9, 11, 13,
15],
    'weights':     ['uniform', 'distance'],
    'metric':      ['euclidean', 'manhattan']
}

grid_search = GridSearchCV(
    KNeighborsClassifier(),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)
grid_search.fit(X_train_scaled, y_train)

print("Best Parameters:",
grid_search.best_params_)
print(f"Best CV Accuracy:
{grid_search.best_score_ * 100:.2f}%")

# Evaluate best model
best_knn = grid_search.best_estimator_
y_pred =
best_knn.predict(X_test_scaled)
acc = accuracy_score(y_test, y_pred)

print("\n========== KNN (With Tuning)
==========")
print(f"Accuracy: {acc * 100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test,
y_pred, target_names=le.classes_))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d',
cmap='Blues',
            xticklabels=le.classes_,
yticklabels=le.classes_)
plt.title('KNN – Confusion Matrix
(Tuned)')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
plt.show()

# Accuracy vs K plot
k_values = range(1, 21)
test_accs = []
for k in k_values:
    knn_k =
KNeighborsClassifier(n_neighbors=k)
    knn_k.fit(X_train_scaled, y_train)
    test_accs.append(knn_k.score(X_test
_scaled, y_test))

plt.figure(figsize=(8, 5))
plt.plot(k_values, test_accs, marker='s',
color='steelblue', label='Test Accuracy')
plt.xlabel('Number of Neighbors (K)')
plt.ylabel('Accuracy')
plt.title('KNN: Test Accuracy vs. K')
plt.xticks(k_values)
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```





10. Conclusion

The KNN experiment on the Kaggle Iris dataset demonstrates that a properly scaled, distance-based classifier can achieve strong performance (93.33% test accuracy) with minimal configuration on a clean, low-dimensional dataset. Iris-setosa is perfectly classified (F1=1.00) owing to its clear linear separability, while the two misclassifications occur exclusively between Iris-versicolor and Iris-virginica — species known to overlap in petal and sepal feature space. This reflects an irreducible Bayes error in the dataset rather than a failure of the algorithm.

GridSearchCV exhaustively explored 32 combinations of K, distance metric, and weighting scheme across 5-fold cross-validation, confirming that the default configuration (K=5, Euclidean distance, uniform weights) is already near-optimal for this problem. The high cross-validation accuracy of 96.67% provides confidence in the model's generalisation ability, while the identical before/after test performance reinforces that the defaults were well-chosen. The experiment also highlights two foundational principles for KNN in practice: feature scaling via StandardScaler is non-negotiable for distance-based methods, and the choice of K requires empirical validation rather than guesswork — even when the optimal K happens to be the default.